

UNIVERSIDAD POLITÉCNICA DE TLAXCALA

Ingeniería en Tecnologías De La Información

Materia: Tecnologías y aplicaciones en internet

Tema: Reporte de Backend y conexión con fronted

Profesor: Máxima Sánchez Cuateta

Alumnos: Erick Rivaldo Flores Maza

Ramses Ortega Ramirez

Oscar Juarez Loaiza

Angel Hakim Vargas Angeles

Raul Alfonso Paredes Oropeza

Grupo: 8 "A"

Contenido

Contenido.....	1
Resumen ejecutivo.....	7
Cronología consolidada.....	8
Cómo leer los estados.....	8
Escala revisada	8
1. Origen, propósito y alcance del producto.....	9
Desarrollo realizado	9
Integración entre capas	9
2. Consolidación del monorepo y etapas de evolución	10
Desarrollo realizado	10
Integración entre capas	10
3. Arquitectura general y separación de responsabilidades.....	11
Desarrollo realizado	11
Integración entre capas	11
4. Fundación del backend Express	12
Desarrollo realizado	12
Integración entre capas	12
5. Autenticación, sesión y recuperación de acceso	13
Desarrollo realizado	13
Integración entre capas	13
6. Persistencia, modelos y estrategia de store	14
Desarrollo realizado	14
Integración entre capas	14
7. Usuarios, roles y autorización RBAC	15
Desarrollo realizado	15
Integración entre capas	15
8. Unidades, conductores y consistencia operativa	16
Desarrollo realizado	16

Integración entre capas	16
9. Rutas, asignación y geometría	17
Desarrollo realizado	17
Integración entre capas	17
10. Seguimiento GPS e integridad temporal.....	18
Desarrollo realizado	18
Integración entre capas	18
11. Mapa móvil y experiencia de seguimiento	19
Desarrollo realizado	19
Integración entre capas	19
12. Jornadas, control, checklist e historial.....	20
Desarrollo realizado	20
Integración entre capas	20
13. Incidencias y flujo SOS	21
Desarrollo realizado	21
Integración entre capas	21
14. Alertas, notificaciones y retroalimentación.....	22
Desarrollo realizado	22
Integración entre capas	22
15. Chat: conversaciones, mensajes y estados.....	23
Desarrollo realizado	23
Integración entre capas	23
16. Cifrado E2EE y protección de conversaciones directas	24
Desarrollo realizado	24
Integración entre capas	24
17. Multimedia, notas de voz y archivos de chat.....	25
Desarrollo realizado	25
Integración entre capas	25
18. Radio PTT y máquina de estado.....	26
Desarrollo realizado	26

Integración entre capas	26
19. Presencia y estado de conexión.....	27
Desarrollo realizado	27
Integración entre capas	27
20. Llamadas WebRTC y señalización	28
Desarrollo realizado	28
Integración entre capas	28
21. Servicio de correo y comunicaciones salientes.....	29
Desarrollo realizado	29
Integración entre capas	29
22. Migración y arquitectura de la aplicación móvil.....	30
Desarrollo realizado	30
Integración entre capas	30
23. Navegación móvil, deep links y políticas de acceso.....	31
Desarrollo realizado	31
Integración entre capas	31
24. Perfil móvil, edición y seguridad de cuenta	32
Desarrollo realizado	32
Integración entre capas	32
25. Operación offline, reconexión y recuperación	33
Desarrollo realizado	33
Integración entre capas	33
26. Ventas: landing pública y descubrimiento comercial	34
Desarrollo realizado	34
Integración entre capas	34
27. Catálogo de planes y motor comercial	35
Desarrollo realizado	35
Integración entre capas	35
28. Checkout, validación y creación de orden	36
Desarrollo realizado	36

Integración entre capas	36
29. Integración con Mercado Pago e idempotencia	37
Desarrollo realizado	37
Integración entre capas	37
30. Suscripción, cambios y cancelación	38
Desarrollo realizado	38
Integración entre capas	38
31. Activación, llaves y alta de conductores	39
Desarrollo realizado	39
Integración entre capas	39
32. Onboarding de empresa y puesta en marcha	40
Desarrollo realizado	40
Integración entre capas	40
33. Arquitectura del Portal web	41
Desarrollo realizado	41
Integración entre capas	41
34. Dashboard de Operaciones	42
Desarrollo realizado	42
Integración entre capas	42
35. Mapa y seguimiento en el Portal	43
Desarrollo realizado	43
Integración entre capas	43
36. Administración de Equipo, Unidades y Rutas	44
Desarrollo realizado	44
Integración entre capas	44
37. Documentos e Incidencias en el Portal	45
Desarrollo realizado	45
Integración entre capas	45
38. Plan, facturación, pagos y perfil de cuenta	46
Desarrollo realizado	46

Integración entre capas	46
39. Mobile App Center, versiones y descarga.....	47
Desarrollo realizado	47
Integración entre capas	47
40. Sistema visual, componentes compartidos y refactor UI	48
Desarrollo realizado	48
Integración entre capas	48
41. Seguridad, privacidad y endurecimiento productivo.....	49
Desarrollo realizado	49
Integración entre capas	49
42. Observabilidad, auditoría y trazabilidad	50
Desarrollo realizado	50
Integración entre capas	50
43. Pruebas automatizadas, typecheck y calidad	51
Desarrollo realizado	51
Integración entre capas	51
44. CI, builds y proceso de release.....	52
Desarrollo realizado	52
Integración entre capas	52
45. Despliegue e infraestructura.....	53
Desarrollo realizado	53
Integración entre capas	53
Conclusiones, riesgos y siguiente etapa	54
Logros consolidados.....	54
Riesgos pendientes	54
Recomendación	54
Fuentes internas principales	54

El índice es un campo automático de Word. Al abrir el archivo, seleccione Actualizar tabla para recalcular números de página definitivos.

Resumen ejecutivo

ManeComb evolucionó en un periodo corto desde una base de gestión de flotilla hasta un ecosistema que combina operación móvil, administración web, venta de planes, activación de organizaciones y comunicación en tiempo real. El repositorio actual reúne cuatro áreas principales: backend, mobile, ventas y communication-service. El inventario local contabiliza 154 archivos en backend, 271 en Mobile, 87 en Ventas y 37 en el servicio de comunicación, además de documentación, scripts e infraestructura.

El backend concentra autenticación, usuarios, unidades, rutas, ubicación, jornadas, incidencias, documentos, notificaciones, comercial, chat, radio, llamadas, aplicación y observabilidad. Mobile ejecuta seguimiento, control, incidencias, chat, Radio PTT, llamadas, perfil y actualización. Ventas integra landing, catálogo, checkout, Mercado Pago, onboarding y Portal. communication-service protege el envío de correo frente a fallos de proveedores y colas.

Frente	Resultado principal	Estado resumido
Operación	Seguimiento, mapa, jornadas, incidencias y alertas conectados.	Implementado; validación física pendiente en escenarios específicos.
Comunicación	Chat E2EE, multimedia, Radio PTT, presencia y WebRTC.	Implementado con certificaciones y pruebas adicionales requeridas en red real.
Comercial	Planes, checkout, Mercado Pago, suscripción y activación.	Certificado a nivel de contratos, pruebas y UI.
Portal	Cuenta, equipo, unidades, rutas, operación, pagos y soporte.	Certificado en estado posterior a auditorías intermedias.

El resultado global es una plataforma integrada con una base técnica sólida, acompañada por deuda explícita: pruebas manuales en dispositivo, verificación de Mongo productivo, cobertura UI del Portal, rendimiento de recuperación de audio y monitoreo real de proveedores.

Cronología consolidada

Etapa	Trabajo predominante	Resultado
31 mayo	Baseline del monorepo y limpieza inicial.	Estructura común para todos los frentes.
3-18 junio	Migración móvil, Ventas, CORS, auth, producción y CI.	Clientes conectados al backend y despliegues estabilizados.
Finales de junio	Comunicación, radio, E2EE, presencia y llamadas.	Tiempo real y seguridad de comunicación fortalecidos.
Primera mitad de julio	RBAC, control, historial, rutas y operación.	Integridad de datos y consistencia operacional.
15-17 julio	Auditorías RC, Portal, comercial, tracking y App Center.	Hallazgos, correcciones y certificaciones por módulo.
18-21 julio	Seguimiento, polilíneas, UI y descarga de app.	Afinamiento visual y conexión de distribución móvil.

Cómo leer los estados

Los RC son fotografías de momentos distintos. Un documento puede declarar un bloqueo que después fue corregido. Este reporte mantiene ambos datos: identifica el hallazgo original, describe la respuesta posterior y utiliza el árbol actual como referencia final. No se considera resuelto aquello que únicamente compila cuando el propio RC exige prueba física o acceso a infraestructura productiva.

Escala revisada

- 131 revisiones Git en el historial analizado.
- Más de 500 archivos entre los cuatro frentes principales.
- 56 archivos de prueba localizados entre Backend y Mobile, más pruebas de comunicación.
- Decenas de reportes RC y auditorías especializadas.
- Despliegues y dependencias externas que requieren verificación por entorno.

1. Origen, propósito y alcance del producto

ManeComb nació como una plataforma para concentrar la operación diaria de flotillas de transporte colectivo. El problema inicial combinaba información dispersa, seguimiento manual de unidades, comunicación por canales no integrados y poca trazabilidad sobre rutas, incidencias y decisiones. La solución evolucionó hacia un monorepo que reúne API, aplicación móvil, experiencia comercial, portal administrativo y servicios de comunicación.

Desarrollo realizado

- Centralización de usuarios, roles, unidades, rutas y operación.
- Seguimiento geográfico y temporal conectado con incidencias y comunicación.
- Canal comercial para compra, activación, facturación y acceso al Portal.
- Aplicación móvil para conductores y operación en campo.
- Backend unificado para contratos REST, Socket.IO, persistencia y seguridad.

Integración entre capas

El alcance no se limita a una aplicación de mapa ni a una tienda de planes. Ventas inicia la relación con la organización; el backend convierte la compra en orden, suscripción y activación; el Portal permite administrar la cuenta y la flotilla; y la app móvil ejecuta la operación diaria. Esta cadena explica por qué los cambios comerciales, de autorización y de seguimiento deben mantenerse sincronizados.

Dimensión	Evidencia del reporte
Frontend	La interfaz consume stores, hooks y contratos; no sustituye reglas autoritativas del servidor.
Backend	Las rutas validan identidad, permisos y datos antes de delegar en servicios y store.
Persistencia / realtime	Los cambios se conservan o distribuyen según el dominio, con segmentación y trazabilidad.
Fuentes verificadas	docs/alcance-sistema-combis.md; docs/project-master.md; historial Git desde el baseline del 31/05/2026

Estado al corte. Producto integrado con frentes comercial, administrativo y operativo activos.

2. Consolidación del monorepo y etapas de evolución

El repositorio se estableció formalmente como monorepo y después atravesó ciclos de estabilización, migración móvil, rediseño comercial, endurecimiento productivo y certificaciones por dominio. El historial visible contiene 131 revisiones hasta el corte analizado, con una concentración significativa de trabajo durante junio y julio de 2026.

Desarrollo realizado

- Baseline y eliminación inicial de código confirmado como muerto.
- Migración del cliente móvil hacia React Native CLI.
- Correcciones de CORS, variables de entorno y despliegue web.
- Rediseño de Ventas y compra de planes SaaS.
- Ciclos RC para comunicación, seguimiento, portal, autorización y release.

Integración entre capas

La evolución fue incremental: primero se construyó la base funcional; después se estabilizaron accesos y producción; luego se profundizó en consistencia operacional, comunicación y experiencia del Portal. Los documentos RC registran hallazgos intermedios que no deben confundirse con el estado final: por ejemplo, una auditoría detectó pantallas faltantes y certificaciones posteriores documentaron su integración.

Dimensión	Evidencia del reporte
Frontend	La interfaz consume stores, hooks y contratos; no sustituye reglas autoritativas del servidor.
Backend	Las rutas validan identidad, permisos y datos antes de delegar en servicios y store.
Persistencia / realtime	Los cambios se conservan o distribuyen según el dominio, con segmentación y trazabilidad.
Fuentes verificadas	git log; RC-RELEASE-CANDIDATE-01.md; RC-PORTAL-FINAL-01.md; múltiples RC de junio-julio

Estado al corte. Evolución trazable por commits y reportes de certificación; quedan validaciones manuales puntuales.

3. Arquitectura general y separación de responsabilidades

La arquitectura actual separa interfaces por contexto sin duplicar la lógica central. Mobile atiende operación en campo; Ventas contiene landing, checkout y Portal; Backend expone la API principal y el socket general; communication-service concentra correo y mensajería saliente tolerante a fallos. MongoDB actúa como persistencia productiva y existe un store embebido para pruebas y desarrollo controlado.

Desarrollo realizado

- REST para operaciones transaccionales y consultas.
- Socket.IO para ubicación, presencia, chat, radio, llamadas e incidencias.
- Zustand para estado de cliente en Mobile y Portal.
- Adaptadores comerciales para separar UI, reglas y proveedores.
- Servicios compartidos de seguridad, telemetría, archivos y notificaciones.

Integración entre capas

La cadena principal es UI -> store/hook -> cliente API -> ruta backend -> store -> respuesta normalizada -> render. En eventos dinámicos, el backend persiste o valida y después emite a salas segmentadas. Esta estructura reduce fuentes paralelas: Portal y Mobile consumen los mismos datos operativos, y el mapa no mantiene un seguimiento independiente del backend.

Dimensión	Evidencia del reporte
Frontend	La interfaz consume stores, hooks y contratos; no sustituye reglas autoritativas del servidor.
Backend	Las rutas validan identidad, permisos y datos antes de delegar en servicios y store.
Persistencia / realtime	Los cambios se conservan o distribuyen según el dominio, con segmentación y trazabilidad.
Fuentes verificadas	backend/src/app.js; backend/src/server.js; ventas/features; mobile/src; communication-service/docs/ARCHITECTURE.md

Estado al corte. Arquitectura modular con backend unificado y servicios auxiliares acotados.

4. Fundación del backend Express

El backend está construido sobre Node.js y Express, con montaje modular de rutas, middlewares transversales y un servidor HTTP compartido con Socket.IO. La base incluye compresión, CORS configurable, Helmet, limitación de solicitudes, trazabilidad por identificador y manejo público de errores.

Desarrollo realizado

- Inicialización de configuración y comprobaciones de preparación.
- Montaje de módulos bajo /api con contratos normalizados.
- Autenticación y autorización mediante middleware.
- Manejo centralizado de 404 y errores no controlados.
- Integración del store embebido o Mongo según disponibilidad y política.

Integración entre capas

La decisión de mantener un backend principal simplifica autenticación, permisos y tiempo real. Chat, GPS, radio y señalización RTC comparten la sesión del socket; las funciones comerciales reutilizan el mismo store y telemetría. La política REQUIRE_MONGO permite impedir degradaciones silenciosas en producción.

Dimensión	Evidencia del reporte
Frontend	La interfaz consume stores, hooks y contratos; no sustituye reglas autoritativas del servidor.
Backend	Las rutas validan identidad, permisos y datos antes de delegar en servicios y store.
Persistencia / realtime	Los cambios se conservan o distribuyen según el dominio, con segmentación y trazabilidad.
Fuentes verificadas	backend/src/app.js; backend/src/server.js; backend/src/config; backend/src/middlewares

Estado al corte. Base productiva endurecida; la disponibilidad final depende de variables y servicios externos.

5. Autenticación, sesión y recuperación de acceso

El módulo de autenticación cubre inicio de sesión, registro permitido por flujo, consulta de sesión, refresh y recuperación de contraseña. El cliente conserva sesión de forma segura y ejecuta inicialización, renovación ante 401 y limpieza coordinada cuando la sesión deja de ser válida.

Desarrollo realizado

- JWT para identidad y autorización de solicitudes.
- Hash de contraseñas con bcrypt y política de fortaleza.
- Promesa singleton de refresh para evitar carreras concurrentes.
- Ruteo por estado de cuenta, rol y suscripción.
- Endurecimiento específico del flujo de recuperación de contraseña.

Integración entre capas

La autenticación conecta el resultado comercial con la operación: no basta con credenciales válidas; el acceso móvil también considera activación y suscripción. En Portal, el layout protege rutas y aplica permisos antes de renderizar consumidores. El respaldo de claves E2EE se vincula al perfil autenticado sin exponerse en respuestas generales.

Dimensión	Evidencia del reporte
Frontend	La interfaz consume stores, hooks y contratos; no sustituye reglas autoritativas del servidor.
Backend	Las rutas validan identidad, permisos y datos antes de delegar en servicios y store.
Persistencia / realtime	Los cambios se conservan o distribuyen según el dominio, con segmentación y trazabilidad.
Fuentes verificadas	backend/src/modules/auth; backend/src/utls/jwt.js; mobile/src/store; RC-AUTHORIZATION-FINAL-01.md; commit 8563da2

Estado al corte. Flujo funcional con refresh, políticas y recuperación endurecida.

6. Persistencia, modelos y estrategia de store

La capa de datos abstrae dos implementaciones: store embebido para desarrollo y pruebas, y MongoDB/Mongoose para persistencia productiva. Los módulos consumen métodos del store mediante `req.app.locals.store`, evitando que las rutas acoplen directamente su lógica a una base específica.

Desarrollo realizado

- Modelos para usuarios, vehículos, rutas, incidencias y conversaciones.
- Persistencia de órdenes, suscripciones, sesiones RTC y eventos.
- GridFS para binarios documentales cuando Mongo está disponible.
- Semillas controladas para entorno local.
- Reglas de integridad y limpieza de asociaciones al eliminar entidades.

Integración entre capas

La misma interfaz de store permite ejecutar pruebas sin infraestructura externa y desplegar con Mongo sin reescribir módulos. No obstante, los datos productivos requieren auditorías específicas: el informe de tracking dejó pendiente contar rutas huérfanas porque el entorno no resolvió DNS, y se evitó inferir resultados inexistentes.

Dimensión	Evidencia del reporte
Frontend	La interfaz consume stores, hooks y contratos; no sustituye reglas autoritativas del servidor.
Backend	Las rutas validan identidad, permisos y datos antes de delegar en servicios y store.
Persistencia / realtime	Los cambios se conservan o distribuyen según el dominio, con segmentación y trazabilidad.
Fuentes verificadas	backend/src/data/models.js; store.js; mongo-store.js; seedData.js; RC-DATA-INTEGRITY-01.md

Estado al corte. Persistencia dual consolidada; producción debe exigir Mongo y auditar datos heredados.

7. Usuarios, roles y autorización RBAC

La gestión de usuarios evolucionó desde cuentas administrativas generales hasta reglas más precisas por rol, organización y permiso. Administradores operan el Portal; supervisores consultan operación; conductores acceden a funciones móviles; compradores gestionan cuenta, plan y activación.

Desarrollo realizado

- Listado, actualización y eliminación con validaciones de integridad.
- Normalización de roles y sanitización de datos sensibles.
- Permisos de Portal para users, vehicles, routes y billing.
- Segmentación por organización en consultas y sockets.
- Alta de conductores mediante activation keys, no creación directa desde Portal.

Integración entre capas

RBAC se aplica en backend y se refleja en navegación. La interfaz no sustituye la autorización del servidor: ocultar una opción mejora la experiencia, pero cada endpoint conserva validación. La eliminación de usuarios limpia o protege asociaciones relevantes para evitar conversaciones, unidades o documentos incongruentes.

Dimensión	Evidencia del reporte
Frontend	La interfaz consume stores, hooks y contratos; no sustituye reglas autoritativas del servidor.
Backend	Las rutas validan identidad, permisos y datos antes de delegar en servicios y store.
Persistencia / realtime	Los cambios se conservan o distribuyen según el dominio, con segmentación y trazabilidad.
Fuentes verificadas	backend/src/modules/users; require-admin.js; ventas/features/portal/utls/access.ts; RC-RBAC-CONTROL-01.md

Estado al corte. Autorización integrada por rol y organización, con controles tanto en API como en Portal.

8. Unidades, conductores y consistencia operativa

El dominio de unidades reúne código, placa, conductor, ruta, estado, capacidad, kilometraje y datos operativos. La evolución reciente se concentró en evitar duplicados, conservar asignaciones y alinear la representación de una misma unidad entre Mobile, Portal y backend.

Desarrollo realizado

- CRUD y consulta de vehículos mediante API.
- Asignación de conductor y ruta con reglas de integridad.
- Prevención de creación duplicada y normalización de identificadores.
- Presentación de código, conductor y ruta en lugar de UUID internos.
- Consistencia del vehículo activo en la aplicación móvil.

Integración entre capas

Las unidades actúan como nodo común: reciben ubicación, participan en jornadas, originan alertas, se asocian a incidencias y aparecen en filtros del mapa. Un hallazgo RC detectó que editar una unidad assigned podía convertirla en available; los ciclos posteriores de consistencia y certificación se orientaron a impedir pérdidas silenciosas de asignación.

Dimensión	Evidencia del reporte
Frontend	La interfaz consume stores, hooks y contratos; no sustituye reglas autoritativas del servidor.
Backend	Las rutas validan identidad, permisos y datos antes de delegar en servicios y store.
Persistencia / realtime	Los cambios se conservan o distribuyen según el dominio, con segmentación y trazabilidad.
Fuentes verificadas	backend/src/modules/vehicles; portal-units-screen.tsx; RC-UNITS-DUPLICATE-CREATE-01.md; RC-MOBILE-VEHICLE-DATA-CONSISTENCY-01.md

Estado al corte. Gestión consolidada con atención especial a integridad de asignaciones.

9. Rutas, asignación y geometría

El sistema administra rutas con nombre, código, color, origen, destino y geometría. La asignación vincula rutas con unidades y alimenta tanto la navegación móvil como la vista operacional del Portal.

Desarrollo realizado

- Creación y edición de rutas con coordenadas validadas.
- Asignación, liberación y duplicación controlada.
- Polilíneas en mapas y miniaturas geométricas.
- Historial de viajes y sesiones por unidad.
- Aislamiento de rutas por organización.

Integración entre capas

La ruta no es solo una figura en el mapa: define contexto para ubicación, jornada, incidentes, checkpoints y métricas. Las mejoras de UX agregaron confirmaciones y evitaron sobrescrituras poco visibles. En backend, crear rutas exige organización para impedir que registros sin tenant se filtren como globales.

Dimensión	Evidencia del reporte
Frontend	La interfaz consume stores, hooks y contratos; no sustituye reglas autoritativas del servidor.
Backend	Las rutas validan identidad, permisos y datos antes de delegar en servicios y store.
Persistencia / realtime	Los cambios se conservan o distribuyen según el dominio, con segmentación y trazabilidad.
Fuentes verificadas	backend/src/modules/navigation; mobile/src/screens/map; portal-routes-screen.tsx; RC-RUTAS-UX-03.md

Estado al corte. Flujo funcional de definición, asignación y visualización con geometría compartida.

10. Seguimiento GPS e integridad temporal

El seguimiento fue uno de los frentes con mayor número de iteraciones. La implementación unificó ubicación enviada por HTTP y Socket, orden temporal, frescura, recuperación y presentación coherente en Mobile y Portal.

Desarrollo realizado

- Normalización de reloj cliente con tolerancia configurable.
- Timestamps de origen, recepción y procesamiento para auditoría.
- Prevención de que paquetes antiguos sobrescriban ubicaciones nuevas.
- gpsFreshness autoritativo compartido por clientes.
- Segmentación de emisiones por organización, rol y usuario.

Integración entre capas

El backend calcula freshUntil y los clientes lo interpretan sin inventar umbrales paralelos. Una ubicación vencida puede conservarse como evidencia histórica, pero no debe mostrarse como posición actual ni adjuntarse a una incidencia como dato fresco. El Portal y Mobile consumen la misma decisión para evitar contradicciones.

Dimensión	Evidencia del reporte
Frontend	La interfaz consume stores, hooks y contratos; no sustituye reglas autoritativas del servidor.
Backend	Las rutas validan identidad, permisos y datos antes de delegar en servicios y store.
Persistencia / realtime	Los cambios se conservan o distribuyen según el dominio, con segmentación y trazabilidad.
Fuentes verificadas	backend/src/services/tracking-time.js; locations/routes.js; mobile/src/screens/map/utills/tracking.ts; RC-TRACKING-EXECUTION-01.md

Estado al corte. Fases técnicas completadas; persisten conteo productivo y matriz manual en dispositivo.

11. Mapa móvil y experiencia de seguimiento

La pantalla de mapa móvil se descompuso en canvas, controles flotantes, HUD, selector de ruta, panel inferior y hooks especializados. Esta separación redujo la concentración de lógica y facilitó pruebas sobre selección, cámara, ubicación y estados operativos.

Desarrollo realizado

- MapCanvas y variantes nativa, web y de tipos compartidos.
- Motor de ubicación y sincronización con reducers y servicios.
- BottomTrackingPanel con información de unidad y jornada.
- Recuperación cuando el mapa o datos no están disponibles.
- Preferencia reduceMotion para omitir animaciones de layout.

Integración entre capas

El mapa combina ruta asignada, posición, jornada, checkpoints e incidentes. La selección debe sobrevivir actualizaciones y cambios de cámara. Las polilíneas se mejoraron en commits recientes, y la vista evita representar como movimiento un dato que no ha sido actualizado.

Dimensión	Evidencia del reporte
Frontend	La interfaz consume stores, hooks y contratos; no sustituye reglas autoritativas del servidor.
Backend	Las rutas validan identidad, permisos y datos antes de delegar en servicios y store.
Persistencia / realtime	Los cambios se conservan o distribuyen según el dominio, con segmentación y trazabilidad.
Fuentes verificadas	mobile/src/screens/map; app-map.native.tsx; commits de polilínea y seguimiento; RC-GEOLOCATION-FORENSIC-AUDIT-01.md

Estado al corte. Arquitectura modular y pruebas focalizadas; certificación física sigue siendo indispensable.

12. Jornadas, control, checklist e historial

La pantalla de control/checklist concentra inicio, pausa, reanudación y cierre de jornada, selección de rutas, puntos de control e historial. El módulo reutiliza contratos de navegación y sesiones, no un backend de checklist completamente separado.

Desarrollo realizado

- Acciones de sesión encapsuladas en route-session-actions.
- Cola o recuperación para operaciones con conectividad limitada.
- Creación y eliminación de rutas guardadas.
- Historial y checkpoints conectados con seguimiento.
- Mensajes y confirmaciones para acciones operativas.

Integración entre capas

Una auditoría RC señaló que no existe una entidad backend checklist autónoma. El diseño actual debe describirse con honestidad: la pantalla funciona sobre navegación, rutas y sesiones. Esto reduce duplicación, pero crea dependencia del modelo operacional y exige pruebas de regresión cuando cambian sus contratos.

Dimensión	Evidencia del reporte
Frontend	La interfaz consume stores, hooks y contratos; no sustituye reglas autoritativas del servidor.
Backend	Las rutas validan identidad, permisos y datos antes de delegar en servicios y store.
Persistencia / realtime	Los cambios se conservan o distribuyen según el dominio, con segmentación y trazabilidad.
Fuentes verificadas	mobile/src/screens/checklist-screen.tsx; mobile/src/services/route-session-actions.ts; RC-CONTROL-CERTIFICATION-01.md

Estado al corte. Flujo operativo amplio, con deuda de definición sobre si checklist debe convertirse en dominio propio.

13. Incidencias y flujo SOS

El módulo de incidencias permite registrar problemas, asociarlos con ruta y unidad, definir severidad y evolucionar el estado de abierta a en proceso o resuelta. Los eventos relevantes se distribuyen en tiempo real y alimentan dashboard y alertas.

Desarrollo realizado

- Creación y listado con permisos por rol y organización.
- Estados y severidades normalizados.
- Evidencias y ubicación condicionada por frescura GPS.
- Notificación de eventos críticos y SOS.
- Pantallas Mobile y Portal para alta, consulta y seguimiento.

Integración entre capas

El flujo conecta ubicación, usuario reportero, unidad y ruta. Una auditoría detectó que el patrón `/^sos/i` podía generar falsos positivos con palabras no relacionadas; el reporte histórico conserva ese hallazgo como evidencia de por qué las reglas de emergencia deben ser exactas y probadas.

Dimensión	Evidencia del reporte
Frontend	La interfaz consume stores, hooks y contratos; no sustituye reglas autoritativas del servidor.
Backend	Las rutas validan identidad, permisos y datos antes de delegar en servicios y store.
Persistencia / realtime	Los cambios se conservan o distribuyen según el dominio, con segmentación y trazabilidad.
Fuentes verificadas	backend/src/modules/incidents; mobile/src/screens/incidents-screen.tsx; portal-incidents-screen.tsx; RC-RELEASE-CANDIDATE-01.md

Estado al corte. Dominio conectado a tiempo real; reglas críticas deben conservar pruebas contra falsos positivos.

14. Alertas, notificaciones y retroalimentación

ManeComb combina notificaciones persistidas, suscripciones push, banners de conexión, mensajes inline y confirmaciones. La auditoría de alertas mostró una base funcional, pero también inconsistencias entre Mobile y Ventas, componentes duplicados y acciones que necesitaban confirmación más clara.

Desarrollo realizado

- Notificaciones dirigidas por usuario o rol con nivel y categoría.
- Deep links hacia chat, radio e incidencias.
- ConnectionBanner para offline, reconexión y sincronización.
- ConfirmModal y Toast en Portal.
- Revisión de lenguaje técnico, acciones destructivas y estados de éxito.

Integración entre capas

Las alertas no deben existir aisladas: deben abrir el objeto que las originó y conservar contexto. La auditoría RC-ALERTS-01 no aprobó la experiencia en su corte por confirmaciones faltantes y mensajes inconsistentes; los trabajos posteriores de UI y operaciones deben leerse como respuesta progresiva, no como negación del hallazgo.

Dimensión	Evidencia del reporte
Frontend	La interfaz consume stores, hooks y contratos; no sustituye reglas autoritativas del servidor.
Backend	Las rutas validan identidad, permisos y datos antes de delegar en servicios y store.
Persistencia / realtime	Los cambios se conservan o distribuyen según el dominio, con segmentación y trazabilidad.
Fuentes verificadas	backend/src/modules/notifications; mobile/src/components/connection-banner.tsx; RC-ALERTS-01.md

Estado al corte. Infraestructura implementada; la coherencia de feedback requiere revisión continua por flujo.

15. Chat: conversaciones, mensajes y estados

El chat evolucionó desde una pantalla concentrada hacia componentes, hooks y utilidades especializadas. Soporta directorio, conversaciones generales y directas, mensajes de texto, adjuntos, voz y estados locales de entrega.

Desarrollo realizado

- ChatHeader, ChatComposer, ChatScreenView y MessageMedia.
- Controlador de conversación y directorio desacoplados.
- Estados pending, sent, delivered, read y failed en la experiencia.
- Reintento de mensajes y continuidad ante desconexión.
- Acciones de llamada y radio integradas en el encabezado.

Integración entre capas

Los mensajes se persisten mediante REST y se retransmiten por Socket.IO. El cliente usa identificadores locales para evitar duplicados y mantiene estados visibles. La refactorización también estandarizó acciones del encabezado y corrigió problemas de teclado, scroll y similitud de render.

Dimensión	Evidencia del reporte
Frontend	La interfaz consume stores, hooks y contratos; no sustituye reglas autoritativas del servidor.
Backend	Las rutas validan identidad, permisos y datos antes de delegar en servicios y store.
Persistencia / realtime	Los cambios se conservan o distribuyen según el dominio, con segmentación y trazabilidad.
Fuentes verificadas	backend/src/modules/chat; mobile/src/screens/chat; RC-CHAT-FINAL-02.md; RC-CHAT-MESSAGE-STATUS-UX-01.md

Estado al corte. Chat modular, multimedia y conectado a presencia, llamadas y cifrado directo.

16. Cifrado E2EE y protección de conversaciones directas

Las conversaciones directas incorporan cifrado de extremo a extremo con TweetNaCl. Las claves y respaldos se asocian a la sesión del usuario, mientras el backend conserva contenido cifrado y metadatos necesarios para entrega.

Desarrollo realizado

- Generación y manejo de claves en cliente.
- Cifrado y descifrado para conversaciones directas.
- Respaldo E2EE protegido por endpoints de autenticación.
- Pruebas unitarias de utilidades criptográficas.
- Sanitización para evitar exponer backups sensibles.

Integración entre capas

E2EE convive con estados, multimedia y persistencia, pero limita qué puede inspeccionar el servidor. La documentación distingue privacidad de transporte y autorización: cifrar contenido no sustituye validar participantes, organización, acceso a conversación ni seguridad del dispositivo.

Dimensión	Evidencia del reporte
Frontend	La interfaz consume stores, hooks y contratos; no sustituye reglas autoritativas del servidor.
Backend	Las rutas validan identidad, permisos y datos antes de delegar en servicios y store.
Persistencia / realtime	Los cambios se conservan o distribuyen según el dominio, con segmentación y trazabilidad.
Fuentes verificadas	mobile/src/utils/chat-e2ee.ts; backend/src/utils/chat-crypto.js; commit 2830ac4; pruebas chat-e2ee

Estado al corte. Cifrado directo conectado; requiere mantener gestión de claves y recuperación bajo pruebas de regresión.

17. Multimedia, notas de voz y archivos de chat

El chat admite audio, imagen y video mediante servicios de carga, reproducción y presentación específicos. La aplicación maneja selección de archivos, previsualización, errores de reproducción y visualización a pantalla completa.

Desarrollo realizado

- Endpoints de media y audio por conversación.
- Abstracciones nativas de selector, imagen, video y audio.
- MessageMedia para renderizar contenido según tipo.
- Transcripción opcional cuando existe proveedor configurado.
- Controles de tamaño, acceso y almacenamiento.

Integración entre capas

La multimedia depende de storage y autorización de conversación. Los fallos de proveedor no deben derribar el flujo de texto. En Radio, los audios también se conservan como mensajes, pero una auditoría identificó una búsqueda $O(N*M)$ que requiere paginación o acceso directo por identificador para escalar.

Dimensión	Evidencia del reporte
Frontend	La interfaz consume stores, hooks y contratos; no sustituye reglas autoritativas del servidor.
Backend	Las rutas validan identidad, permisos y datos antes de delegar en servicios y store.
Persistencia / realtime	Los cambios se conservan o distribuyen según el dominio, con segmentación y trazabilidad.
Fuentes verificadas	backend/src/services/chat-media.js; audio-transcription.js; mobile/src/screens/chat/components/message-media.tsx

Estado al corte. Soporte multimedia implementado; rendimiento de recuperación de audios debe vigilarse.

18. Radio PTT y máquina de estado

Radio PTT es un subsistema operativo separado de las llamadas. La experiencia móvil utiliza reducir de sesión, servicios de audio y tiempo real, waveform, tarjetas de transmisión y estados explícitos para captura, envío, reproducción y error.

Desarrollo realizado

- Mantener presionado para transmitir con retroalimentación háptica.
- Arbitraje de canal activo mediante Redis en la evolución reciente.
- Servicios independientes para audio y eventos realtime.
- Persistencia de transmisiones y reproducción desde historial.
- Auditorías específicas de SSOT, render, carreras y lifecycle.

Integración entre capas

Radio comparte autenticación, conversación y socket, pero no comparte ciclo de vida ni foreground service con llamadas. Esta separación evita que un cambio de WebRTC afecte la reproducción PTT. Los reportes de radio documentan la consolidación de una sola fuente de verdad y la eliminación de estados duplicados.

Dimensión	Evidencia del reporte
Frontend	La interfaz consume stores, hooks y contratos; no sustituye reglas autoritativas del servidor.
Backend	Las rutas validan identidad, permisos y datos antes de delegar en servicios y store.
Persistencia / realtime	Los cambios se conservan o distribuyen según el dominio, con segmentación y trazabilidad.
Fuentes verificadas	mobile/src/screens/radio; backend/src/modules/radio; docs/radio-ssot-audit; commit 73275cc

Estado al corte. Subsistema maduro y auditado, con dependencia operacional de audio, red y Redis según despliegue.

19. Presencia y estado de conexión

La presencia informa si usuarios y dispositivos están conectados y evita inferencias inconsistentes en chat, radio y operación. Un ciclo específico consolidó connection state como fuente única de verdad.

Desarrollo realizado

- Eventos de conexión y desconexión por Socket.IO.
- Salas por usuario, rol y organización.
- Indicadores de presencia en UI.
- Manejo de reconexión y epoch de sesión.
- Pruebas de utilidades de presencia y realtime-state.

Integración entre capas

La presencia no equivale a entrega de mensaje ni a ubicación fresca. El reporte separa estos conceptos: un usuario puede estar conectado sin GPS reciente, y un mensaje enviado al servidor puede no estar entregado al dispositivo. Mantener estados independientes evita promesas falsas en la UI.

Dimensión	Evidencia del reporte
Frontend	La interfaz consume stores, hooks y contratos; no sustituye reglas autoritativas del servidor.
Backend	Las rutas validan identidad, permisos y datos antes de delegar en servicios y store.
Persistencia / realtime	Los cambios se conservan o distribuyen según el dominio, con segmentación y trazabilidad.
Fuentes verificadas	mobile/src/components/presence-indicator.tsx; mobile/src/utls/presence.ts; commit 6188d6e; RC-PRESENCE-FINAL-01.md

Estado al corte. Estado de conexión consolidado y reutilizado por comunicación.

20. Llamadas WebRTC y señalización

Las llamadas 1 a 1 usan el socket general para señalización y WebRTC P2P para medios. STUN/TURN resuelve conectividad, mientras el backend valida acceso a la sala y conserva sesiones RTC como historial.

Desarrollo realizado

- Eventos join, leave, offer, answer, ICE y hangup.
- Ack de ingreso con busy y forbidden.
- Foreground service Android para micrófono y cámara.
- Timer de gracia de 15 segundos ante desconexión.
- Configuración TURN estática o coturn REST.

Integración entre capas

La arquitectura rechazó un microservicio y socket adicional porque la escala actual no los justificaba. Chat es dueño de la conexión y llamadas la consume. El CDR reutiliza el store existente; las funciones grupales, grabación y SFU permanecen fuera del alcance actual.

Dimensión	Evidencia del reporte
Frontend	La interfaz consume stores, hooks y contratos; no sustituye reglas autoritativas del servidor.
Backend	Las rutas validan identidad, permisos y datos antes de delegar en servicios y store.
Persistencia / realtime	Los cambios se conservan o distribuyen según el dominio, con segmentación y trazabilidad.
Fuentes verificadas	docs/CALLING-ARCHITECTURE.md; backend/src/sockets/index.js; mobile/src/native/webrtc.ts; RC-WEBRTC-CERTIFICATION-01.md

Estado al corte. Diseño proporcional y funcional; TURN y foreground requieren prueba entre dispositivos y redes reales.

21. Servicio de correo y comunicaciones salientes

communication-service organiza correos transaccionales y otros canales salientes mediante proveedores intercambiables, plantillas, prioridades, colas, métricas e historial. El endurecimiento se enfocó en impedir que DNS, Redis o un proveedor externo derriben Node.js.

Desarrollo realizado

- Providers para Resend, SMTP, SES, Mailgun, Postmark y SendGrid.
- Resultado estructurado success/error en lugar de excepciones no capturadas.
- BullMQ cuando Redis está disponible y cola local como fallback.
- Validación de configuración antes de habilitar proveedor.
- Workers con reintentos y registro de fallo.

Integración entre capas

Los flujos comerciales y de cuenta pueden solicitar correo sin asumir que el proveedor está disponible. Si la cola falla, el servicio intenta envío directo; si el proveedor no está configurado, devuelve error controlado. Esta resiliencia desacopla continuidad del backend de la disponibilidad de terceros.

Dimensión	Evidencia del reporte
Frontend	La interfaz consume stores, hooks y contratos; no sustituye reglas autoritativas del servidor.
Backend	Las rutas validan identidad, permisos y datos antes de delegar en servicios y store.
Persistencia / realtime	Los cambios se conservan o distribuyen según el dominio, con segmentación y trazabilidad.
Fuentes verificadas	communication-service; RC-COMMUNICATION-FINAL-01.md; 16 pruebas de comunicación documentadas

Estado al corte. Módulo crash-proof certificado frente a errores de proveedor, red y cola.

22. Migración y arquitectura de la aplicación móvil

El cliente móvil activo migró desde una etapa basada en Expo hacia React Native CLI. La documentación histórica conserva referencias antiguas, pero el stack de build y despliegue vigente se encuentra en mobile/ con Android nativo y abstracciones multiplataforma.

Desarrollo realizado

- React Native 0.81 y React 19.
- Navegación centralizada en router y registro de rutas.
- Módulos nativos para ubicación, audio, llamadas, imágenes y almacenamiento seguro.
- Android Gradle, servicios foreground y tareas headless.
- Compatibilidad web selectiva mediante archivos .web y .native.

Integración entre capas

La migración obligó a revisar permisos, deep links, teclado, background, build y distribución. El Portal no reemplaza la app: administra la organización, mientras Mobile ejecuta rutas, ubicación, incidencias y comunicación en campo.

Dimensión	Evidencia del reporte
Frontend	La interfaz consume stores, hooks y contratos; no sustituye reglas autoritativas del servidor.
Backend	Las rutas validan identidad, permisos y datos antes de delegar en servicios y store.
Persistencia / realtime	Los cambios se conservan o distribuyen según el dominio, con segmentación y trazabilidad.
Fuentes verificadas	mobile/README.md; docs/mobile-release.md; commit 4435681; mobile/android

Estado al corte. React Native CLI es la fuente vigente; referencias Expo se consideran históricas.

23. Navegación móvil, deep links y políticas de acceso

La navegación móvil se endureció mediante registro de rutas, políticas de acceso, linking y pruebas específicas. El router decide entre autenticación, gate de cuenta y superficies operativas según sesión, rol y estado comercial.

Desarrollo realizado

- Route registry con pruebas de cobertura.
- Deep links hacia chat, radio e incidencias.
- Política de navegación independiente de la UI.
- Protección contra rutas inválidas y estados de bootstrap.
- Infraestructura de inputs y teclado segura.

Integración entre capas

Las notificaciones pueden dirigir a una intención, pero el router vuelve a comprobar acceso y disponibilidad. La navegación nunca debe permitir que un deep link salte autorización. Los estados de cuenta provenientes del backend influyen en el gate y enlazan directamente Ventas con la app operativa.

Dimensión	Evidencia del reporte
Frontend	La interfaz consume stores, hooks y contratos; no sustituye reglas autoritativas del servidor.
Backend	Las rutas validan identidad, permisos y datos antes de delegar en servicios y store.
Persistencia / realtime	Los cambios se conservan o distribuyen según el dominio, con segmentación y trazabilidad.
Fuentes verificadas	mobile/src/navigation; deep-linking.test.ts; navigation-policy.test.ts; account-routing.ts

Estado al corte. Navegación centralizada y probada contra rutas, sesión y cuenta.

24. Perfil móvil, edición y seguridad de cuenta

El perfil móvil presenta identidad, rol, organización, unidad y preferencias, y ofrece edición de datos permitidos. También concentra cierre de sesión, información legal y acceso a configuración relacionada.

Desarrollo realizado

- Pantallas separadas de consulta y edición.
- Avatar e información normalizada del usuario.
- Mensajes de éxito y error al guardar.
- Almacenamiento seguro de sesión y claves.
- Acceso a privacidad, términos y datos de cuenta.

Integración entre capas

Los datos de perfil se consumen en chat, incidencias, radio y asignación. Por ello, la actualización debe refrescar el store sin duplicar identidades. La auditoría de alertas señaló confirmaciones faltantes en cierre de sesión y pérdida de cambios; estas observaciones forman parte de la mejora continua de UX.

Dimensión	Evidencia del reporte
Frontend	La interfaz consume stores, hooks y contratos; no sustituye reglas autoritativas del servidor.
Backend	Las rutas validan identidad, permisos y datos antes de delegar en servicios y store.
Persistencia / realtime	Los cambios se conservan o distribuyen según el dominio, con segmentación y trazabilidad.
Fuentes verificadas	mobile/src/screens/profile-screen.tsx; profile-edit-screen.tsx; secure-store.ts; RC-ALERTS-01.md

Estado al corte. Perfil operativo implementado; acciones destructivas deben conservar confirmación coherente.

25. Operación offline, reconexión y recuperación

La aplicación incorpora caché offline, banners de conexión, reintentos y preservación de trabajo para escenarios móviles. El objetivo no es fingir que todo está actualizado, sino distinguir información local, pendiente y confirmada.

Desarrollo realizado

- offline-cache con pruebas unitarias.
- ConnectionBanner para offline y sincronización.
- Estados pendientes en chat y acciones de jornada.
- Conservación del último mapa válido con frescura visible.
- Session epoch para invalidar respuestas de sesiones anteriores.

Integración entre capas

Cada dominio define su política: un borrador puede guardarse localmente; una ubicación antigua no puede presentarse como actual; una acción operativa pendiente requiere reconciliación; y un mensaje debe reintentarse sin duplicarse. Esta diferenciación protege integridad y confianza.

Dimensión	Evidencia del reporte
Frontend	La interfaz consume stores, hooks y contratos; no sustituye reglas autoritativas del servidor.
Backend	Las rutas validan identidad, permisos y datos antes de delegar en servicios y store.
Persistencia / realtime	Los cambios se conservan o distribuyen según el dominio, con segmentación y trazabilidad.
Fuentes verificadas	mobile/src/api/offline-cache.ts; connection-banner.tsx; session-epoch.ts; realtime-state.ts

Estado al corte. Resiliencia transversal implementada con políticas específicas por dato.

26. Ventas: landing pública y descubrimiento comercial

La experiencia pública de Ventas presenta ManeComb, planes y beneficios, y conduce al checkout. La evolución incluyó rediseño visual, corrección de CORS, URL de API, comportamiento sin conexión y despliegue en Cloudflare.

Desarrollo realizado

- Catálogo obtenido desde GET /api/commercial/plans.
- Cinco planes reales renderizados desde backend.
- Estados de carga, error y reintento separados del vacío real.
- Diseño responsive y movimiento reducido.
- Rutas limpias y shims compatibles con hosting estático.

Integración entre capas

La landing no contiene precios paralelos: consume el catálogo central. Si la API falla, no muestra falsamente que no existen planes. La selección conserva planId hacia checkout, orden, proveedor de pago, suscripción y Portal.

Dimensión	Evidencia del reporte
Frontend	La interfaz consume stores, hooks y contratos; no sustituye reglas autoritativas del servidor.
Backend	Las rutas validan identidad, permisos y datos antes de delegar en servicios y store.
Persistencia / realtime	Los cambios se conservan o distribuyen según el dominio, con segmentación y trazabilidad.
Fuentes verificadas	ventas/src y features/commercial; RC-COMERCIAL-FINAL-01.md; docs/deploy-ventas-cloudflare.md

Estado al corte. Landing conectada a catálogo productivo y preparada para error de API.

27. Catálogo de planes y motor comercial

El backend mantiene un catálogo estático versionado de planes por tamaño de flotilla. El motor comercial separa reglas, tipos, contratos y adaptadores para que UI y proveedor de pago consuman la misma definición.

Desarrollo realizado

- Planes para 2, 4, 6, 8 y 12 combis.
- Addon de Radio según elegibilidad o inclusión.
- Snapshot del plan guardado con la orden.
- Validación de planId y total en backend.
- Cache de planes y adaptadores de experiencia.

Integración entre capas

El catálogo es fuente única para nombres, precios, badges y características. No se agregó CRUD ni modelo Mongo porque no existía necesidad confirmada. El snapshot protege órdenes históricas ante cambios futuros del catálogo.

Dimensión	Evidencia del reporte
Frontend	La interfaz consume stores, hooks y contratos; no sustituye reglas autoritativas del servidor.
Backend	Las rutas validan identidad, permisos y datos antes de delegar en servicios y store.
Persistencia / realtime	Los cambios se conservan o distribuyen según el dominio, con segmentación y trazabilidad.
Fuentes verificadas	backend/src/config/commercial-plans.js; ventas/features/commercial; RC-VENTAS-PLANES-FINAL-01.md

Estado al corte. Catálogo central certificado; descuentos no forman parte del contrato actual.

28. Checkout, validación y creación de orden

El checkout recopila información de empresa, contacto y facturación, valida la selección y crea una orden comercial. La UI usa hooks de experiencia y servicios de validación, mientras el backend recalcula y conserva datos autoritativos.

Desarrollo realizado

- Validación de datos antes de enviar.
- Orden con planId, addons, total y snapshot.
- Estados de envío y recuperación de error.
- Separación de reglas comerciales y componentes visuales.
- Continuidad hacia Mercado Pago o instrucciones aplicables.

Integración entre capas

El cliente nunca es autoridad del importe. El backend valida plan y addons, crea la orden y genera la preferencia. El mismo identificador se mantiene en metadata y external_reference para reconciliar confirmación y webhook.

Dimensión	Evidencia del reporte
Frontend	La interfaz consume stores, hooks y contratos; no sustituye reglas autoritativas del servidor.
Backend	Las rutas validan identidad, permisos y datos antes de delegar en servicios y store.
Persistencia / realtime	Los cambios se conservan o distribuyen según el dominio, con segmentación y trazabilidad.
Fuentes verificadas	ventas/features/commercial/hooks/use-checkout-experience.ts; checkout-validation.ts; backend/src/modules/commercial

Estado al corte. Checkout conectado extremo a extremo con validación del servidor.

29. Integración con Mercado Pago e idempotencia

La integración comercial crea preferencias de Mercado Pago, procesa confirmación y webhook, y activa solo pagos aprobados. Las pruebas cubren credenciales, selección de entorno, metadata, importes y referencias.

Desarrollo realizado

- Preferencia con external_reference y metadata coherentes.
- Separación sandbox/producción.
- Webhook y confirmación idempotentes.
- Addon incluido en el total calculado.
- Protección frente a credenciales ausentes o ambiguas.

Integración entre capas

El pago no activa directamente desde la interfaz. La API recibe evidencia del proveedor, actualiza la orden y deriva suscripción y onboarding. No se ejecutó un cobro real durante certificación para evitar una transacción; el contrato se verificó con proveedor simulado y pruebas de webhook.

Dimensión	Evidencia del reporte
Frontend	La interfaz consume stores, hooks y contratos; no sustituye reglas autoritativas del servidor.
Backend	Las rutas validan identidad, permisos y datos antes de delegar en servicios y store.
Persistencia / realtime	Los cambios se conservan o distribuyen según el dominio, con segmentación y trazabilidad.
Fuentes verificadas	backend/src/services/commercial-payment.js; test/mercado-pago.test.js; RC-COMERCIAL-FINAL-01.md

Estado al corte. Contrato certificado con simulación; monitoreo de webhooks sigue siendo requisito productivo.

30. Suscripción, cambios y cancelación

La suscripción representa el acceso comercial vigente de la organización. Ventas incluye una máquina de presentación para estados como trial, activo, pago fallido, cancelado y cambio programado, alineada con reglas del backend.

Desarrollo realizado

- Activación derivada de pago aprobado.
- Cambio de plan mediante acción real.
- Cancelación con protección contra repetición.
- Presentación de periodo e importe sin prometer renovación inexistente.
- Consistencia del estado activo compartido entre backend, Portal y Mobile.

Integración entre capas

El estado de suscripción condiciona acceso y acciones. Una corrección reciente unificó la interpretación de suscripción activa y evitó que el Mobile derivara acceso de datos inconsistentes. Las transiciones se validan en backend; el frontend traduce a mensajes y acciones permitidas.

Dimensión	Evidencia del reporte
Frontend	La interfaz consume stores, hooks y contratos; no sustituye reglas autoritativas del servidor.
Backend	Las rutas validan identidad, permisos y datos antes de delegar en servicios y store.
Persistencia / realtime	Los cambios se conservan o distribuyen según el dominio, con segmentación y trazabilidad.
Fuentes verificadas	ventas/features/commercial/subscription-state.ts; subscription-validator.ts; commit a1c3daf; RC-SUBSCRIPTION-CONSISTENCY-01.md

Estado al corte. Ciclo comercial real conectado; no se prometen reactivaciones o renovaciones no implementadas.

31. Activación, llaves y alta de conductores

Después de la compra, la organización ingresa a un flujo de activación. El administrador genera llaves para que cada conductor complete su propio registro, evitando que el Portal cree credenciales o perfiles incompletos directamente.

Desarrollo realizado

- Generación, copia, compartir y revocación de activation keys.
- Canje por parte del conductor.
- Validación de unidad y suscripción activa.
- Timeline de eventos de activación.
- Navegación guiada hacia Equipo, Unidades y Rutas.

Integración entre capas

La llave conecta identidad, organización y capacidad contratada. Las correcciones simplificaron el registro y validaron que una unidad no quede asociada de forma inválida. Portal muestra el progreso, mientras Mobile completa el alta del conductor.

Dimensión	Evidencia del reporte
Frontend	La interfaz consume stores, hooks y contratos; no sustituye reglas autoritativas del servidor.
Backend	Las rutas validan identidad, permisos y datos antes de delegar en servicios y store.
Persistencia / realtime	Los cambios se conservan o distribuyen según el dominio, con segmentación y trazabilidad.
Fuentes verificadas	portal-onboarding-screen.tsx; backend módulos de activación/comercial; RC-ACTIVATION-FLOW-CONSOLIDATION-01.md

Estado al corte. Flujo consolidado y certificado en Portal, con responsabilidades separadas por actor.

32. Onboarding de empresa y puesta en marcha

El onboarding guía desde la suscripción aprobada hasta una operación mínima: perfil de empresa, equipo, unidades, rutas y acceso móvil. El diseño evita presentar el Portal como terminado solo porque existe el pago.

Desarrollo realizado

- Wizard y progreso por pasos.
- Enlaces directos a pantallas administrativas.
- Starter fleet cuando aplica.
- Seguimiento de eventos y bloqueos.
- Mensajes diferenciados entre pendiente, activo y acción requerida.

Integración entre capas

Cada paso consume datos reales del backend. El onboarding no mantiene una segunda lista de unidades o usuarios: consulta los stores existentes. La llave de conductor y la descarga de la app forman parte de la puesta en marcha, no del checkout mismo.

Dimensión	Evidencia del reporte
Frontend	La interfaz consume stores, hooks y contratos; no sustituye reglas autoritativas del servidor.
Backend	Las rutas validan identidad, permisos y datos antes de delegar en servicios y store.
Persistencia / realtime	Los cambios se conservan o distribuyen según el dominio, con segmentación y trazabilidad.
Fuentes verificadas	portal-onboarding-screen.tsx; commercial-activation.js; RC-PORTAL-ACTIVATION-FINAL-01.md

Estado al corte. Ciclo de activación completo y enlazado con administración real.

33. Arquitectura del Portal web

El Portal vive dentro del paquete Ventas y utiliza React Native Web/Vite para ofrecer administración de cuenta y operación. PortalLayout protege el acceso, organiza navegación y aplica permisos; stores separados mantienen datos operativos y comerciales.

Desarrollo realizado

- Layout responsive con sidebar y navegación por secciones.
- useAppStore para usuarios y unidades.
- usePortalStore para cuenta, plan, facturas, sesiones y onboarding.
- Cliente API compartido y normalizador de errores.
- Error boundary a nivel de aplicación.

Integración entre capas

El Portal reutiliza endpoints existentes; no implementa lógica de negocio móvil ni crea fuentes paralelas. La certificación final verificó la cadena UI -> store -> API -> backend -> store -> render y el control de permisos por módulo.

Dimensión	Evidencia del reporte
Frontend	La interfaz consume stores, hooks y contratos; no sustituye reglas autoritativas del servidor.
Backend	Las rutas validan identidad, permisos y datos antes de delegar en servicios y store.
Persistencia / realtime	Los cambios se conservan o distribuyen según el dominio, con segmentación y trazabilidad.
Fuentes verificadas	ventas/features/portal/components/portal-layout.tsx; use-portal-store.ts; RC-PORTAL-FINAL-01.md

Estado al corte. Portal certificado dentro de configuración y credenciales del entorno.

34. Dashboard de Operaciones

La pantalla principal del Portal concentra métricas, jornadas, eventos, checkpoints, posiciones y accesos rápidos. Fue una de las superficies con más iteraciones de UI y refactor por su densidad funcional.

Desarrollo realizado

- Filtros por unidad, conductor, ruta y estado.
- Métricas derivadas de datos operativos reales.
- Paneles de historial, recorridos y detalle.
- Mapa OperationsMap conectado a ubicaciones y geometría.
- Acciones rápidas reutilizando PortalButton.

Integración entre capas

El dashboard no utiliza fixtures: combina usuarios, unidades y endpoints de historial. Las mejoras sustituyeron UUID por etiquetas comprensibles, tradujeron estados de jornada y normalizaron errores. También se corrigieron fallos de runtime durante extracciones de componentes compartidos.

Dimensión	Evidencia del reporte
Frontend	La interfaz consume stores, hooks y contratos; no sustituye reglas autoritativas del servidor.
Backend	Las rutas validan identidad, permisos y datos antes de delegar en servicios y store.
Persistencia / realtime	Los cambios se conservan o distribuyen según el dominio, con segmentación y trazabilidad.
Fuentes verificadas	portal-dashboard-screen.tsx; operations-map.tsx; RC-OPERATIONS-UI-MASTER-01.md; RC-PORTAL-OPERATIONS-CERTIFICATION-01.md

Estado al corte. Centro operativo consolidado y conectado a seguimiento real.

35. Mapa y seguimiento en el Portal

El mapa web permite observar unidades, recorridos y rutas sin exponer configuración técnica al usuario. Si Mapbox falta o falla, la experiencia conserva una representación funcional basada en última ubicación real y datos de recorrido.

Desarrollo realizado

- OperationsMap con marcadores y geometría.
- Fallback operativo sin instrucciones para desarrolladores.
- Filtros completos sin truncar resultados.
- Frescura GPS compartida con Mobile.
- Detalle de jornada, ruta y conductor.

Integración entre capas

Portal y Mobile usan la misma ubicación autoritativa. El mapa web no calcula una verdad alterna y respeta gpsFreshness. Las mejoras recientes rediseñaron seguimiento y polilíneas, con énfasis en estilos y lectura de recorrido.

Dimensión	Evidencia del reporte
Frontend	La interfaz consume stores, hooks y contratos; no sustituye reglas autoritativas del servidor.
Backend	Las rutas validan identidad, permisos y datos antes de delegar en servicios y store.
Persistencia / realtime	Los cambios se conservan o distribuyen según el dominio, con segmentación y trazabilidad.
Fuentes verificadas	ventas/features/portal/components/operations-map.tsx; utils/tracking.ts; RC-MAP-SALES-DEEP-AUDIT-01.md

Estado al corte. Seguimiento web funcional con fallback y paridad de datos.

36. Administración de Equipo, Unidades y Rutas

Tres pantallas cubren las entidades principales de puesta en marcha. Equipo administra usuarios habilitados; Unidades gestiona flotilla y asignaciones; Rutas define y vincula recorridos.

Desarrollo realizado

- Listas con estados de carga, vacío y error.
- Formularios y modales de edición.
- Confirmaciones para eliminar, liberar o revocar.
- Permisos users, vehicles y routes.
- Listas compartidas mediante PortalDataList y PortalDataRow.

Integración entre capas

Las pantallas consumen la misma identidad de unidad y usuario que Operaciones. La extracción de PortalButton y primitivas de lista redujo repetición, pero se verificó runtime debido a un incidente previo donde estilos evaluados al cargar el módulo derribaron Operaciones aun cuando typecheck pasaba.

Dimensión	Evidencia del reporte
Frontend	La interfaz consume stores, hooks y contratos; no sustituye reglas autoritativas del servidor.
Backend	Las rutas validan identidad, permisos y datos antes de delegar en servicios y store.
Persistencia / realtime	Los cambios se conservan o distribuyen según el dominio, con segmentación y trazabilidad.
Fuentes verificadas	portal-users-screen.tsx; portal-units-screen.tsx; portal-routes-screen.tsx; portal-button.tsx; portal-data-list.tsx

Estado al corte. Administración integrada con componentes compartidos y reglas de acceso.

37. Documentos e Incidencias en el Portal

Una auditoría de release detectó que el backend ya soportaba documentos e incidencias mientras el Portal no ofrecía superficies administrativas. El estado posterior del repositorio incluye portal-documents-screen y portal-incidents-screen, por lo que el reporte registra la brecha como etapa histórica y la incorporación como evolución posterior.

Desarrollo realizado

- Listado y composición de filas con primitivas compartidas.
- Consulta de estados y acciones disponibles.
- Integración con contratos existentes del backend.
- Manejo de contenido real, carga y error.
- Separación entre funcionalidad consolidada y pantallas aún en afinamiento.

Integración entre capas

Documentos se relaciona con perfil, revisión y vencimientos; incidencias con seguimiento, unidad y alertas. La incorporación al Portal cerró una brecha administrativa, aunque las reglas visuales y de carga documental requieren validación continua y no deben documentarse con supuestos no verificados.

Dimensión	Evidencia del reporte
Frontend	La interfaz consume stores, hooks y contratos; no sustituye reglas autoritativas del servidor.
Backend	Las rutas validan identidad, permisos y datos antes de delegar en servicios y store.
Persistencia / realtime	Los cambios se conservan o distribuyen según el dominio, con segmentación y trazabilidad.
Fuentes verificadas	RC-RELEASE-CANDIDATE-01.md; portal-documents-screen.tsx; portal-incidents-screen.tsx; backend modules

Estado al corte. Pantallas presentes en el estado actual; su madurez debe evaluarse por flujo y pruebas runtime.

38. Plan, facturación, pagos y perfil de cuenta

El Portal ofrece superficies para consultar suscripción, facturas, pagos, empresa, seguridad y soporte. Estas pantallas consumen la orden y suscripción persistidas en lugar de inventar una billetera local.

Desarrollo realizado

- Mi plan con estado, importe y acciones reales.
- Facturas y descarga mediante URL del backend.
- Pagos y reintento sobre el plan existente.
- Perfil de empresa y sesiones activas.
- Revocación de sesión remota con confirmación.

Integración entre capas

La certificación comercial eliminó métodos de pago locales que Mercado Pago no consumía y mensajes que prometían renovaciones inexistentes. Los errores deben distinguir ausencia real de datos de fallo de carga para no alarmar a clientes con suscripción válida.

Dimensión	Evidencia del reporte
Frontend	La interfaz consume stores, hooks y contratos; no sustituye reglas autoritativas del servidor.
Backend	Las rutas validan identidad, permisos y datos antes de delegar en servicios y store.
Persistencia / realtime	Los cambios se conservan o distribuyen según el dominio, con segmentación y trazabilidad.
Fuentes verificadas	portal-plan-screen.tsx; portal-billing-screen.tsx; portal-payments-screen.tsx; portal-profile-screen.tsx

Estado al corte. Cuenta y ciclo comercial conectados al backend; feedback de errores continúa como área sensible.

39. Mobile App Center, versiones y descarga

El Mobile App Center conecta configuración de versión en backend, administración en Portal, aviso de actualización en Mobile y descarga pública de la aplicación. Los commits más recientes enlazaron frontend y backend para la descarga.

Desarrollo realizado

- GET y PATCH de información de versión.
- Estadísticas de versiones de dispositivos.
- UpdateBanner y updateInfo en store móvil.
- Pantalla Portal para administrar versión y artefacto.
- Descarga expuesta desde experiencia comercial.

Integración entre capas

La fuente de verdad es store.getAppConfig. Portal refleja GET /app/info y Mobile recibe updateInfo durante login/refresh. Una certificación arquitectónica corrigió un import wrapErrors inexistente que podía causar crash en startup y alineó requireAdmin con el patrón de middleware.

Dimensión	Evidencia del reporte
Frontend	La interfaz consume stores, hooks y contratos; no sustituye reglas autoritativas del servidor.
Backend	Las rutas validan identidad, permisos y datos antes de delegar en servicios y store.
Persistencia / realtime	Los cambios se conservan o distribuyen según el dominio, con segmentación y trazabilidad.
Fuentes verificadas	RC-MOBILE-APP-CENTER-05.2-CERTIFICATION.md; portal-app-movil-screen.tsx; update-banner.tsx; commits 71f88fd-30a2052

Estado al corte. Módulo certificado y conectado de administración a descarga y actualización.

40. Sistema visual, componentes compartidos y refactor UI

Mobile y Portal evolucionaron desde implementaciones locales hacia componentes reutilizables. El Portal introdujo PortalButton y primitivas de lista; Mobile mantiene PrimaryButton, StatusPill, AppCard y componentes de shell.

Desarrollo realizado

- Tokens de tema y paleta por superficie.
- PortalButton con variantes, tamaños, iconos, loading y disabled.
- PortalDataList y PortalDataRow con zonas composables.
- Cards, badges y layouts compartidos.
- Revisión de runtime además de typecheck y build.

Integración entre capas

La lección principal fue que compilación no garantiza seguridad de evaluación de módulo. PortalButton provocó un error runtime al resolver estilos antes de inicialización; la corrección trasladó resolución al render. Las extracciones posteriores mantuvieron esa restricción y migraron solo patrones que encajaban limpiamente.

Dimensión	Evidencia del reporte
Frontend	La interfaz consume stores, hooks y contratos; no sustituye reglas autoritativas del servidor.
Backend	Las rutas validan identidad, permisos y datos antes de delegar en servicios y store.
Persistencia / realtime	Los cambios se conservan o distribuyen según el dominio, con segmentación y trazabilidad.
Fuentes verificadas	ventas/features/portal/components; portal-theme.ts; mobile/src/components; historial de extracciones

Estado al corte. Base compartida en crecimiento, con disciplina de verificar cada pantalla en ejecución.

41. Seguridad, privacidad y endurecimiento productivo

La seguridad se trabajó como capa transversal: autenticación, autorización, aislamiento de organización, headers, CORS, rate limiting, secretos, sanitización y almacenamiento seguro. También se corrigieron problemas productivos de proxy y variables.

Desarrollo realizado

- Helmet, CORS configurable y express-rate-limit.
- JWT, bcrypt y política de contraseñas.
- requireAdmin y permisos por dominio.
- Aislamiento multi-tenant en rutas y sockets.
- E2EE en chat directo y secure store en móvil.

Integración entre capas

El despliegue en Render requirió confiar correctamente en proxy y manejar secretos mínimos. Cloudflare y API deben compartir orígenes autorizados. La seguridad del frontend reduce exposición, pero la decisión final permanece en backend. Los datos sensibles se eliminan de respuestas sanitizadas.

Dimensión	Evidencia del reporte
Frontend	La interfaz consume stores, hooks y contratos; no sustituye reglas autoritativas del servidor.
Backend	Las rutas validan identidad, permisos y datos antes de delegar en servicios y store.
Persistencia / realtime	Los cambios se conservan o distribuyen según el dominio, con segmentación y trazabilidad.
Fuentes verificadas	backend/src/middlewares; config/env.js; RC-AUTHORIZATION-FINAL-01.md; RC-CLOUDFLARE-VITE-ENV-01.md

Estado al corte. Controles principales activos; secretos, CORS y tenant deben auditarse por entorno.

42. Observabilidad, auditoría y trazabilidad

El backend registra eventos de API, solicitudes lentas, errores, actividad comercial, push, RTC e incidencias. Los identificadores de traza permiten correlacionar solicitudes y el Portal administrativo consulta snapshots operativos.

Desarrollo realizado

- x-trace-id en solicitudes.
- Eventos estructurados por módulo, acción y estado.
- Auditoría de cambios administrativos.
- Métricas de socket y sesiones RTC.
- Historial comercial y de activación.

Integración entre capas

La observabilidad se mantuvo proporcional: llamadas reutilizan logger y store en vez de introducir Prometheus o tracing distribuido específico. Communication-service registra fallos estructurados sin derribar procesos. La trazabilidad permite relacionar compra, activación, sesión y operación.

Dimensión	Evidencia del reporte
Frontend	La interfaz consume stores, hooks y contratos; no sustituye reglas autoritativas del servidor.
Backend	Las rutas validan identidad, permisos y datos antes de delegar en servicios y store.
Persistencia / realtime	Los cambios se conservan o distribuyen según el dominio, con segmentación y trazabilidad.
Fuentes verificadas	backend/src/services/telemetry.js; audit.js; ops module; RC-WEBRTC-CERTIFICATION-01.md

Estado al corte. Trazabilidad integrada; conviene establecer retención y tableros productivos formales.

43. Pruebas automatizadas, typecheck y calidad

El repositorio contiene pruebas unitarias e integración distribuidas principalmente en backend y Mobile. En el conteo actual se localizaron 29 archivos de prueba en backend, 27 en Mobile y una prueba en communication-service; Ventas depende especialmente de typecheck, build y smokes específicos.

Desarrollo realizado

- Backend: contratos, tracking, comercial, auth y servicios.
- Mobile: navegación, mapa, radio, offline, E2EE y utilidades.
- TypeScript estricto en Mobile y Ventas.
- Build Vite y Android assembleDebug en certificaciones.
- Smokes de Mercado Pago, API, CORS y comunicación.

Integración entre capas

Los reportes distinguen pruebas automáticas de validación manual. Una RC de tracking documentó 21 suites y 99 pruebas Mobile verdes, suite backend verde y builds aprobados, pero mantuvo abierta la certificación por falta de dispositivo y acceso a Mongo productivo. Esta separación evita declarar producción solo porque compila.

Dimensión	Evidencia del reporte
Frontend	La interfaz consume stores, hooks y contratos; no sustituye reglas autoritativas del servidor.
Backend	Las rutas validan identidad, permisos y datos antes de delegar en servicios y store.
Persistencia / realtime	Los cambios se conservan o distribuyen según el dominio, con segmentación y trazabilidad.
Fuentes verificadas	backend/test; mobile/src/**/*.*.test.ts; RC-TRACKING-EXECUTION-01.md; RC-CI-REPAIR-01.md

Estado al corte. Cobertura significativa en capas críticas; Portal necesita ampliar pruebas automatizadas de UI/runtime.

44. CI, builds y proceso de release

El proceso de release combina typecheck, pruebas, builds web y Android, revisión de diferencias y smokes. Diversos ciclos repararon CI, URLs de API, configuración Android y compatibilidad de despliegue.

Desarrollo realizado

- npm run typecheck en clientes TypeScript.
- npm test en paquetes con suites definidas.
- Vite build para Ventas.
- Gradle assembleDebug/assembleRelease para Android.
- Checklist y reportes de release candidate.

Integración entre capas

El release no es un único comando: requiere validar backend, Portal, Mobile y proveedores. Los errores concurrentes se documentan sin atribuirlos al cambio en evaluación. Las advertencias de chunk grande en Vite se registran como informativas, no como build fallido.

Dimensión	Evidencia del reporte
Frontend	La interfaz consume stores, hooks y contratos; no sustituye reglas autoritativas del servidor.
Backend	Las rutas validan identidad, permisos y datos antes de delegar en servicios y store.
Persistencia / realtime	Los cambios se conservan o distribuyen según el dominio, con segmentación y trazabilidad.
Fuentes verificadas	.github; package.json por paquete; RC-CI-REPAIR-01.md; RC-RELEASE-01.md

Estado al corte. Proceso repetible con certificaciones; falta automatizar más pruebas end-to-end del Portal.

45. Despliegue e infraestructura

La solución contempla despliegue de backend en Render, Ventas en Cloudflare, MongoDB para persistencia, Redis para colas/arbitraje cuando aplica y coturn para llamadas en redes restrictivas. Docker Compose documenta dependencias locales y productivas.

Desarrollo realizado

- Variables VITE_API_URL y orígenes CORS.
- REQUIRE_MONGO para impedir store volátil en producción.
- Redis para BullMQ y arbitraje de Radio según configuración.
- TURN_URLS, TURN_SECRET y TURN_REALM para coturn REST.
- Artefactos Android y descarga desde el ecosistema comercial.

Integración entre capas

Cada dependencia debe degradar de forma explícita. Correo puede deshabilitarse sin crash; TURN ausente reduce llamadas a STUN; Mongo requerido debe detener el arranque si no está disponible; y el Portal debe mostrar fallback de mapa sin filtrar variables técnicas.

Dimensión	Evidencia del reporte
Frontend	La interfaz consume stores, hooks y contratos; no sustituye reglas autoritativas del servidor.
Backend	Las rutas validan identidad, permisos y datos antes de delegar en servicios y store.
Persistencia / realtime	Los cambios se conservan o distribuyen según el dominio, con segmentación y trazabilidad.
Fuentes verificadas	docker-compose.yml; docker-compose.prod.yml; docs/deployment.md; docs/deploy-ventas-cloudflare.md

Estado al corte. Arquitectura de despliegue definida; certificación final exige credenciales y pruebas del entorno real.

Conclusiones, riesgos y siguiente etapa

El desarrollo realizado demuestra una transición desde funciones aisladas hacia un sistema con contratos compartidos. La compra se convierte en suscripción y activación; la activación habilita perfiles, unidades y rutas; la operación produce ubicación, jornadas e incidencias; y la comunicación conecta alertas, chat, radio y llamadas. El valor principal está en esa continuidad.

Logros consolidados

- Backend modular con persistencia Mongo/embebida y Socket.IO.
- App React Native CLI con operación, mapas y comunicación.
- Ventas y Portal conectados a catálogo, pago, suscripción y administración reales.
- Comunicación resiliente frente a fallos de proveedores y red.
- Auditorías y RC que registran decisiones, correcciones y límites honestos.

Riesgos pendientes

- Completar matrices manuales en dispositivos físicos y redes distintas.
- Verificar datos productivos que no pudieron auditarse por restricciones de acceso.
- Aumentar pruebas automatizadas de UI y runtime en el Portal.
- Monitorear rendimiento de radio, retención de eventos y disponibilidad de proveedores.
- Mantener documentación alineada con el estado actual, separando histórico de certificación final.

Recomendación

La siguiente fase debe priorizar certificación operacional reproducible: escenarios end-to-end con cuentas y datos controlados, evidencias de dispositivo, observabilidad de producción y criterios de aceptación por rol. Las nuevas funciones deben integrarse a contratos existentes y demostrar su necesidad antes de crear otro store, servicio o fuente de verdad.

Fuentes internas principales

Código vigente de backend/, mobile/, ventas/ y communication-service/; docs/project-master.md; docs/alcance-sistema-combis.md; documentación de despliegue y llamadas; historial Git; reportes RC de release, Portal, Comercial, Tracking, Comunicación, App Center, autorización, radio, presencia y UI.